



Operating Systems

Unit 1 – Part 1

Subject Code: CI42

B.E Sem: IV

Credits : 4



Syllabus

Unit I

Introduction: What operating systems do, Computer System organization, Computer System architecture, Operating System structure, Operating System operations. **Operating system structures:** Operating system services, user operating system, Interface, System calls, Types of system calls, Operating system structure, System boot.

What is an Operating System?

Definition:

- An Operating System (OS) is a program that manages the computer's hardware.
- It acts as an **intermediary** between the computer user and the computer hardware
- OS is a **resource allocator**
 - Manages all resources
 - Decides between conflicting requests for efficient and fair resource use
- OS is a **control program**
 - Controls execution of programs to prevent errors and improper use of the computer

What is an Operating System?

The Three Main Roles:

- **Hardware Manager:** Controls and coordinates the use of hardware (CPU, memory, I/O devices) among various applications.
- **System Allocator:** Manages resources to prevent conflicts and ensure efficiency.
- **Control Program:** Controls the execution of user programs to prevent errors and improper use of the computer.

The Kernel:

- The one program running at all times on the computer is called the kernel.
- Everything else is either a system program (ships with the OS) or an application program.

What is an Operating System?

Primary Goals of an OS:

1. **Execute User Programs:** Make solving user problems easier.
2. **Convenience (User View):** Make the computer system easy to use (primary goal for PCs and mobile devices).
3. **Efficiency (System View):** Ensure computer hardware is used in an efficient manner (primary goal for mainframes and shared servers).



Computer System Components

A Computer System is divided into four main components:

1. Hardware:

- Provides the basic computing resources.
- Examples: Central Processing Unit (CPU), memory, and Input/Output (I/O) devices.

2. Operating System:

- Controls and coordinates the use of hardware among various applications and users.
- Acts as the "government" of the system, providing the environment for other programs to do useful work.

3. Application Programs:

- Define the ways in which system resources are used to solve specific computing problems for users.
- Examples: Word processors, compilers, web browsers, database systems, video games.

4. Users:

- Can be people, machines, or other computers.



Computer System Components

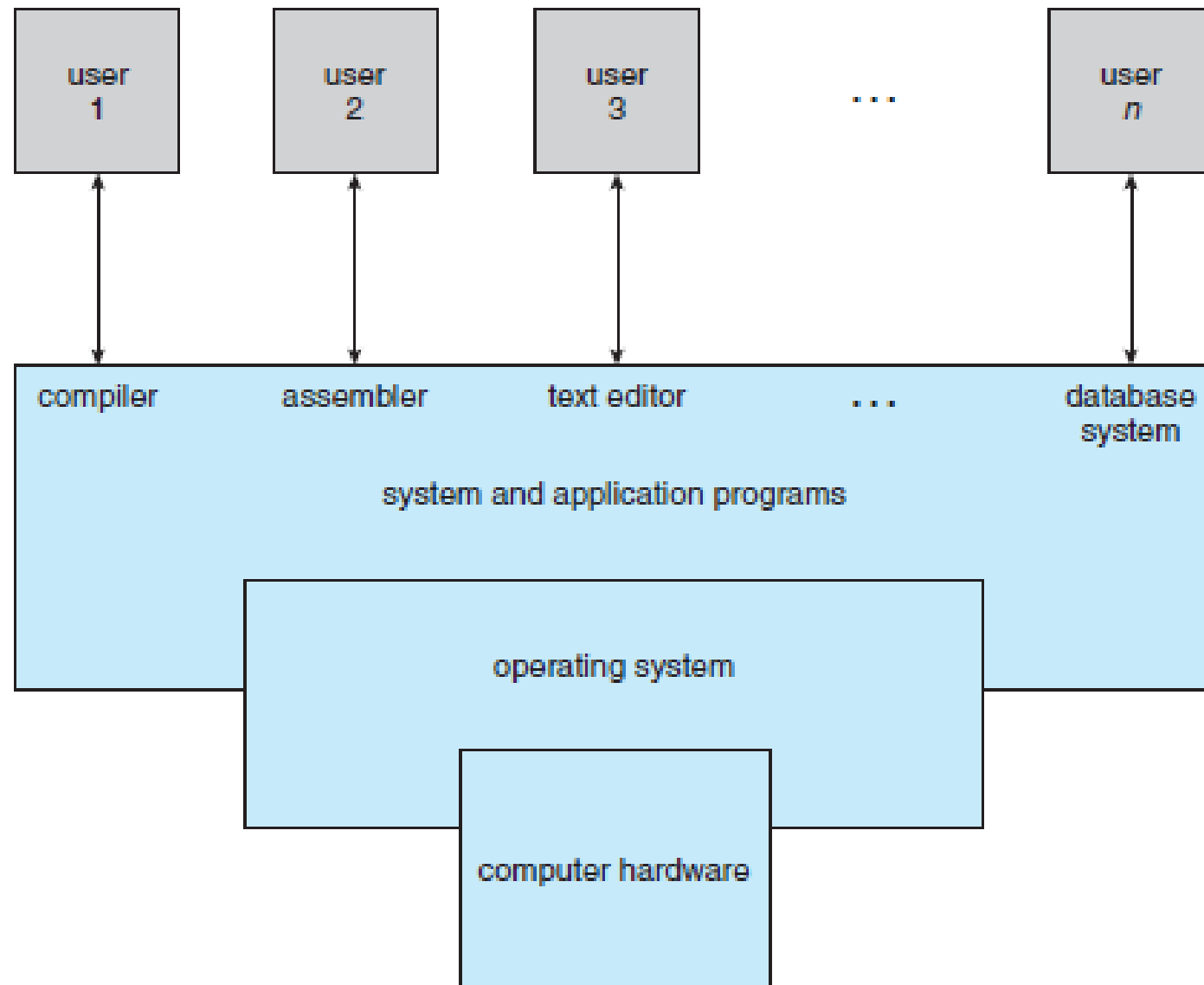


Figure 1.1 Abstract view of the components of a computer system.



What Operating Systems Do

The OS Role Depends on the Point of View:

The function of an OS changes based on who is using the computer and the type of hardware being managed.

1. PC / Desktop Users (User View):

- **Goal:** Maximize **convenience**, **ease of use**, and **performance**.
- **Resource Utilization:** Users **don't care** about efficient resource utilization (e.g., CPU/memory waste) because the system is dedicated to one person.

2. Shared Computers (Mainframe/Minicomputer):

- **Goal:** **Efficiency** and **Fairness**.
- The OS must keep **all users happy** by maximizing resource utilization (CPU time, memory, I/O) so no single user monopolizes the system.

3. Workstations (Dedicated Systems):

- Users have dedicated resources (CPU/RAM) but frequently execute tasks using **shared resources** from servers (networking, printing, file storage).
- **Goal:** Compromise between individual usability and efficient resource sharing.



4. Handheld/Mobile Computers:

- **Constraint:** Often **resource-poor** compared to PCs (slower processors, less memory).
- **Goal:** Optimized for **usability** (touchscreens) and **battery life**.

5. Embedded Computers:

- Found in home devices (toasters, thermostats) and **automobiles**.
- **Interface:** Have **little or no user interface** (numeric keypads or indicator lights).
- **Goal:** Designed primarily to run without user intervention.



The user's perspective is defined by the **Interface** they use. The primary goal is to make the computer easy to use.

- **PC/Desktop Users:**
 - **Goal:** Ease of use and performance.
 - **Resource Utilization:** Often ignored. Because one person "monopolizes" the system, the OS doesn't care if the CPU is idle; it cares that the user's experience is smooth.
- **Mobile Devices (Phones/Tablets):**
 - **Goal:** Usability (touch interfaces) and power management.
 - **Constraint:** Performance is balanced against **battery life** and wireless connectivity.
- **Embedded Systems (IoT/Cars):**
 - **Goal:** Operation without intervention. Often have little to no "user view" or UI; they simply run a specific task reliably.



The System View: Focus on Efficiency

The system sees the OS as a **Resource Allocator** and a **Control Program**.

- **Resource Allocator:**
 - Manages conflicting requests for the **CPU time, Memory space, and File storage**.
 - Decides how to share resources fairly and efficiently among many users/programs.
- **Control Program:**
 - Manages the execution of user programs to prevent errors.
 - Prevents improper use of the computer and controls I/O (Input/Output) devices.
- **Mainframes/Servers:**
 - The goal is **Maximum Resource Utilization**. The OS ensures the expensive hardware is never sitting idle while users are waiting in a queue.



The Initialization Problem:

When a computer is powered up or rebooted, the main memory (RAM) is empty or in an unknown state. The CPU needs an initial program to run immediately.

The Solution: Bootstrap Program:

- **Definition:** The very first code executed by the CPU upon power-up.
- **Storage (Firmware):**
 - Typically stored in **ROM** (Read-Only Memory) or **EEPROM** (Electrically Erasable Programmable ROM).
 - It is known as **firmware** because it is software embedded in hardware and is "immutable" (cannot be easily changed like normal disk files).

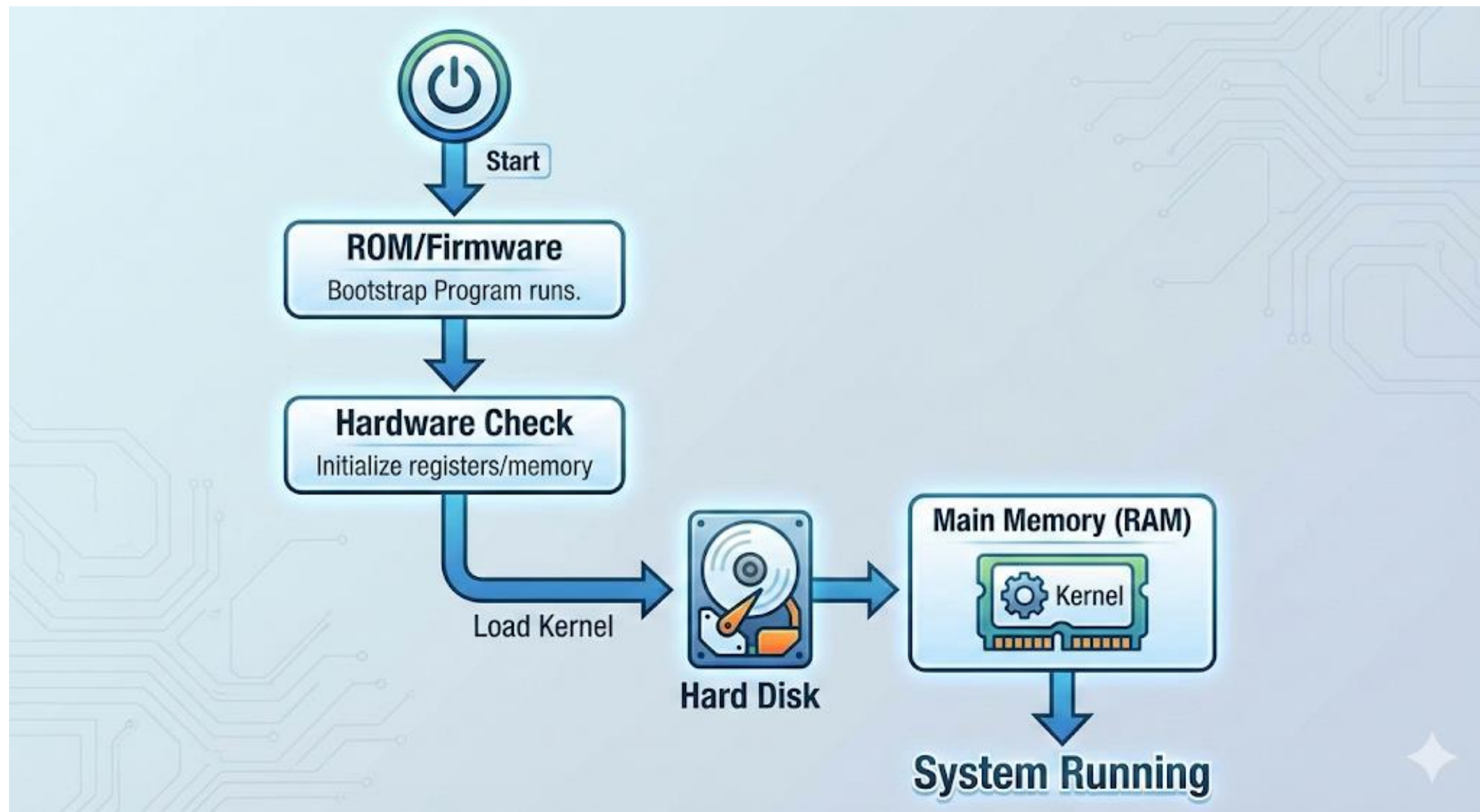


Key Responsibilities:

- **Hardware Initialization:** Initializes all aspects of the system, including CPU registers, device controllers, and memory contents.
- **Diagnostics:** Performs a "Power-On Self-Test" (POST) to ensure hardware (RAM, keyboard, drives) is functioning correctly.
- **OS Loading:** Locates the Operating System kernel on the storage device (disk), loads it into main memory, and starts its execution.

Modern Booting (Two-Step Process):

Complex OSs (Windows, Linux) use a two-step process. The simple ROM bootstrap loads a more complex **Boot Loader** (e.g., GRUB) from the disk (usually Block 0/MBR), which then loads the full OS kernel.





Computer-System Organization

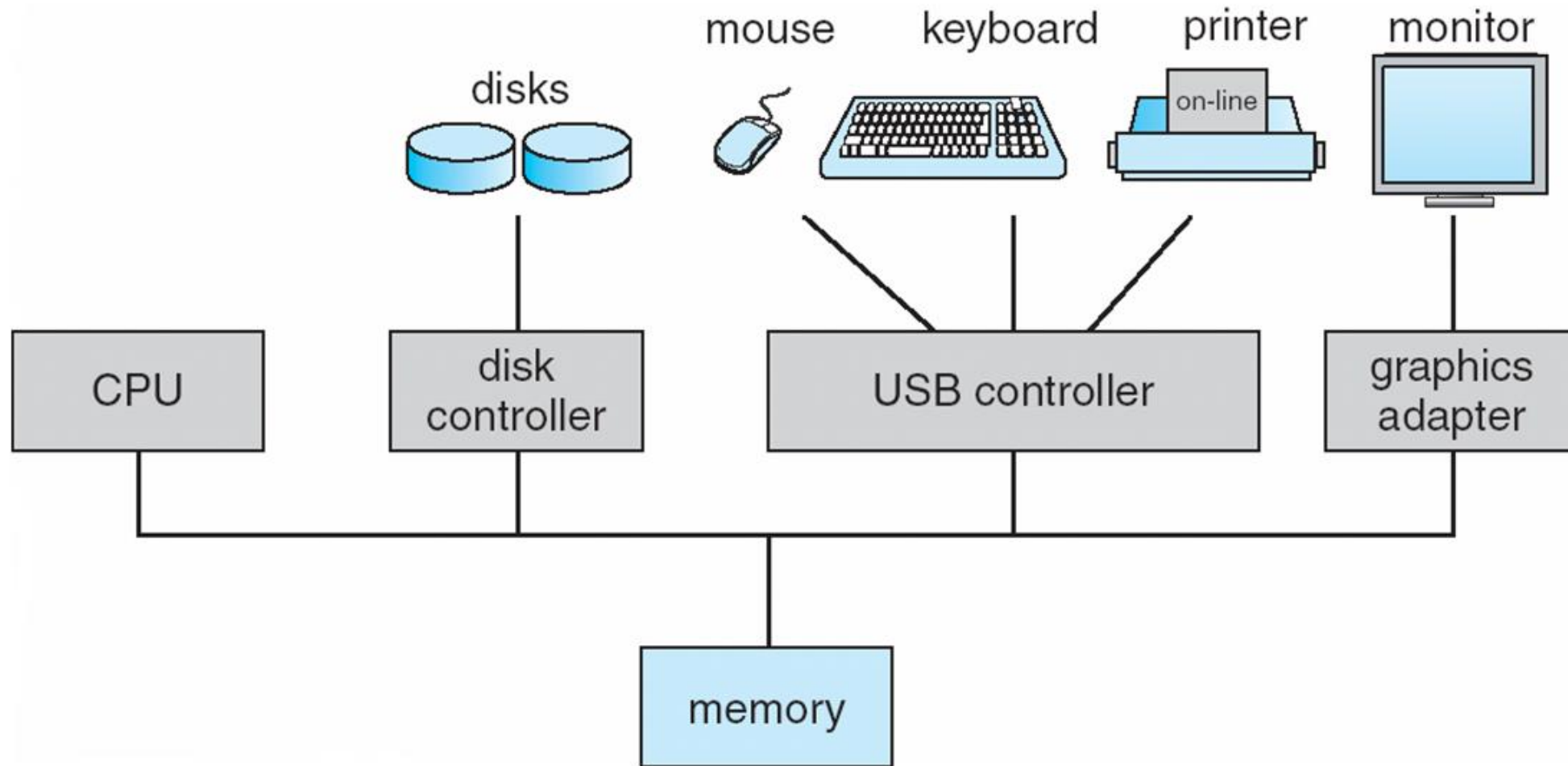
- Computer-System Operation
- Storage Structure
- I/O Structure



Basic Structure:

- A modern general-purpose computer consists of **one or more CPUs** and multiple **device controllers** (e.g., disk controller, USB controller, graphics adapter).
- All components connect through a **common bus** that provides access to **shared memory** (RAM).
- **Device Controllers:**
 - Each device controller is in charge of a specific type of device (e.g., disk drives, audio devices).
- **Concurrent Execution:**
 - The CPU and the device controllers can execute **in parallel**.
 - **Example:** The CPU can be performing calculations for a game while the disk controller is simultaneously loading the next level from the hard drive.
- **Memory Contention:**
 - Since both the CPU and I/O devices need to access RAM, they **compete for memory cycles**.
 - A **Memory Controller** synchronizes access to the shared memory to ensure orderly access and prevent collisions.

Computer-System Operation





Concurrent Execution:

- I/O devices and the CPU can execute **concurrently**.
- While the CPU is processing instructions (like running a calculation), the disk drive can be reading data simultaneously.

• Device Controllers:

- Each device controller is in charge of a particular device type (e.g., Disk Controller, Audio Controller, Video Controller).
- **Local Buffer:** Each device controller has a local buffer (a small amount of fast memory on the card itself).

• Data Movement Lifecycle:

1. **I/O from Device:** I/O occurs from the external device (like the spinning disk) to the **local buffer** of the controller.
2. **Interrupt:** The device controller informs the CPU that it has finished its operation (data is ready in the buffer) by causing an **interrupt**.
3. **CPU Action:** The CPU moves data from/to main memory to/from the local buffers.



Common Functions of Interrupts

An operating system is **interrupt driven**. If there are no processes to execute or I/O devices to service, the OS sits quietly and waits for an event.

Interrupt Handling Mechanism:

- **Control Transfer:** An interrupt transfers control to the **interrupt service routine (ISR)** generally through the **interrupt vector**, which contains the addresses of all the service routines.
- **Saving State:** The interrupt architecture *must* save the address of the interrupted instruction (usually on the stack or in a register) so the CPU can return to it later.

Concurrency Control:

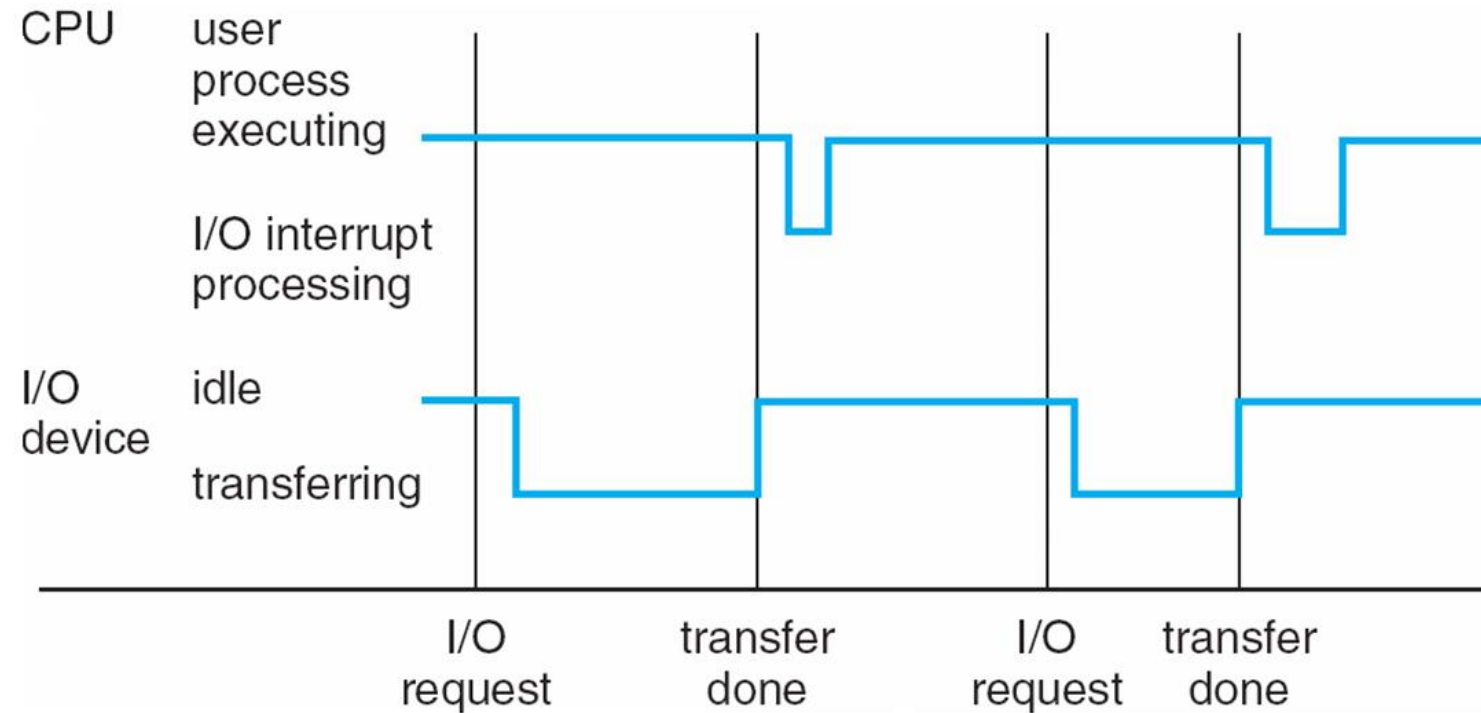
- **Disabling Interrupts:** Incoming interrupts are often **disabled** while another interrupt is being processed to prevent a lost interrupt (ensures critical updates happen without interference).

Traps (Software Interrupts):

- A **trap** (or exception) is a software-generated interrupt caused either by an **error** (e.g., division by zero) or a **user request** (e.g., a system call).



Common Functions of Interrupts





Preserving CPU State:

- The operating system preserves the state of the CPU by storing **registers** and the **program counter (PC)**.
- This ensures that when the interrupt processing is complete, the interrupted computation can resume exactly where it left off, as if nothing happened.

Determining Interrupt Type:

The OS must determine which device caused the interrupt. Two methods are used:

- 1. Polling:** A generic routine queries each device one by one to see which one needs service.
- 2. Vectored Interrupt System:** The device sends a unique ID (vector) with the interrupt request. The OS uses this ID as an index into an **interrupt vector** (a table of pointers) to find the specific address of the service routine.

Action Execution:

- Separate segments of code (Interrupt Service Routines) determine what action should be taken for each type of interrupt.



Main Memory (RAM):

- **CPU Dependency:** The CPU can load instructions *only* from memory, so any program to run must be stored there.
- **Technology:** Most general-purpose computers use **Dynamic Random-Access Memory (DRAM)**.

Specialized Memory (Firmware):

- **ROM (Read-Only Memory):** Cannot be changed; stores static programs like the **bootstrap program**.
- **EEPROM (Electrically Erasable Programmable Read-Only Memory):** Can be changed infrequently; used in smartphones to store factory-installed programs.



The Volatility Problem:

Ideally, programs and data would reside in main memory permanently, but this is impossible for two reasons:

- **Size:** Main memory is usually too small to store all needed programs and data.
- **Volatility:** RAM loses its contents when power is turned off.



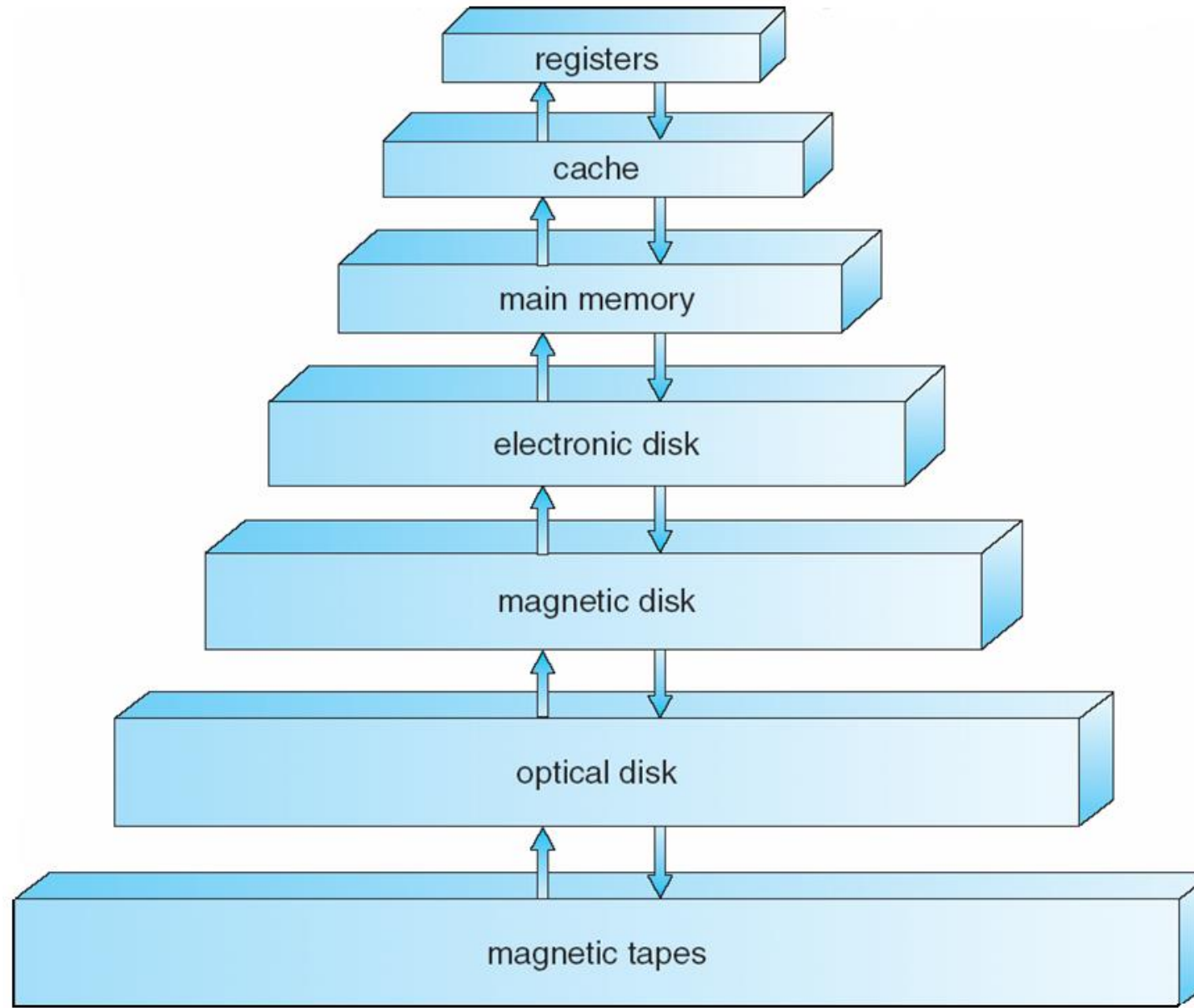
Secondary Storage:

- **Definition:** An extension of main memory able to hold large quantities of data permanently.
- **Requirement:** Must be **nonvolatile** (retains data without power).

Secondary Storage Types:

- **Magnetic Disks:** The most common secondary storage. Used as both source and destination of processing. Proper management is central to OS design.
- **Solid-State Disks (SSD):**
 - Generally faster than magnetic disks and are nonvolatile.
 - **Flash Memory:** Popular in mobile devices/robots; slower than DRAM but needs no power to retain contents.
 - **NVRAM:** DRAM with battery backup power; as fast as DRAM but nonvolatile (as long as the battery lasts).

Storage-Device Hierarchy





Storage-Device Hierarchy

Storage systems are organized in a hierarchy based on **speed**, **cost**, and **volatility**.

- **Higher Levels:** Expensive, fast, small capacity.
- **Lower Levels:** Inexpensive, slow, large capacity.

The Hierarchy Levels (Top to Bottom):

1. **Registers** (Fastest/Volatile)
2. **Cache**
3. **Main Memory**
4. **Solid-State Disk (SSD)**
5. **Magnetic Disk**
6. **Optical Disk**
7. **Magnetic Tapes** (Slowest/Nonvolatile).

Volatility Boundary:

- **Volatile:** Registers, Cache, and Main Memory lose data when power is removed.
- **Nonvolatile:** Solid-state disks and everything below retain data without power.



Device Management Components:

Device Controller:

- Hardware component in charge of a specific device type (e.g., disk controller, SCSI controller).
- Maintains **local buffer storage** and a set of special-purpose **registers**.
- Responsible for moving data between the peripheral device and its local buffer.

Device Driver:

- Operating System software that understands the specific device controller.
- Provides a **uniform interface** to the rest of the OS, hiding the hardware details.



Standard I/O Operation (Interrupt-Driven):

1. **Start:** Driver loads registers in the controller to request an action (e.g., "Read character").
2. **Transfer:** Controller moves data from the device to its local buffer.
3. **Interrupt:** Once the buffer is full, the controller sends an interrupt to the CPU.
4. **Completion:** Driver returns control/data to the OS.

Limitation: Fine for small data but produces high CPU overhead for bulk data (one interrupt per byte).

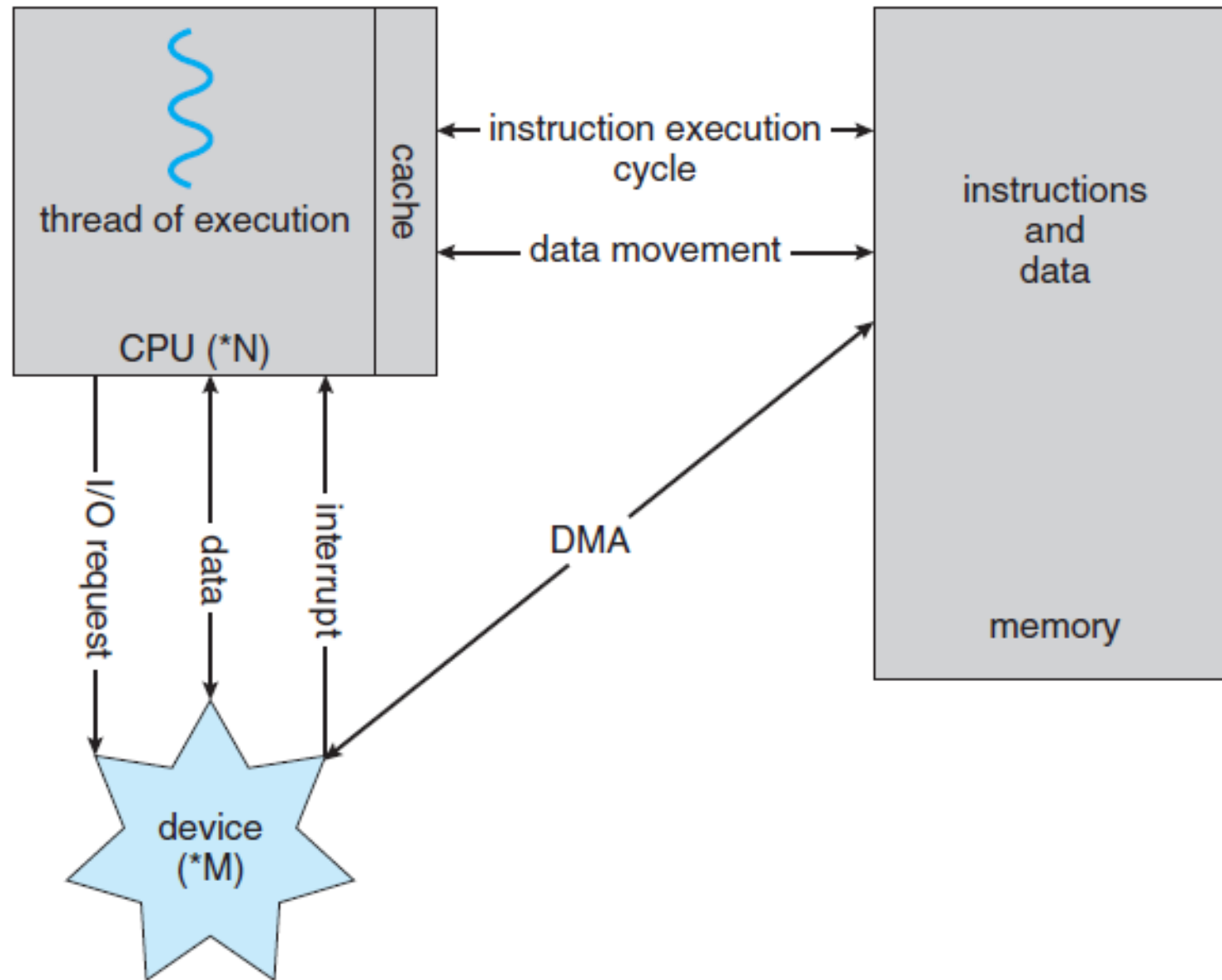


Direct Memory Access (DMA)

- **Purpose:** Solves the overhead problem for high-speed devices (e.g., disk drives).
- **Mechanism:** The device controller transfers an **entire block of data** directly to/from main memory without CPU intervention.
- **Efficiency:** Only **one interrupt** is generated per block, rather than one interrupt per byte.
- **Benefit:** The CPU is free to execute other work while the device controller is transferring data.



How a modern computer system works





Computer-System Architecture

Computer systems are categorized roughly according to the **number of general-purpose processors** used.

1. **Single-Processor Systems**
2. **Multiprocessor Systems (Parallel / Multicore)**



Definition:

- A Single-Processor System typically has **one main CPU**.
- This main CPU is capable of executing a **general-purpose instruction set**, including instructions from user processes.

Special-Purpose Processors:

- Almost all single-processor systems contain other processors.
- **Examples:** Disk controllers, keyboard microprocessors, graphics controllers, and I/O processors.

Key Characteristics of Special Processors:

1. **Limited Scope:** They run a **limited instruction set** and **do not** run user processes.
2. **Specific Tasks:** They perform specific jobs (like moving disk data or processing keystrokes) to relieve the main CPU of overhead.
3. **Management:** They may be managed by the OS (receiving tasks/monitoring status) or run autonomously (low-level hardware).



Definition:

- Also known as **Parallel Systems** or **Multicore Systems**.
- Consists of **two or more processors** in close communication.
- They share the computer bus, clock, memory, and peripheral devices.

Three Main Advantages:

1. Increased Throughput:

- **Goal:** Get more work done in less time.

2. Economy of Scale:

- **Cost:** Cheaper than buying multiple separate computer systems.
- **Sharing:** Processors share peripherals, mass storage (disks), and power supplies, reducing hardware costs.

3. Increased Reliability:

- **Graceful Degradation:** If one processor fails, the system does not crash; it just slows down.
- **Fault Tolerance:** The ability to continue operating despite a hardware failure.

The "Boss-Worker" Relationship:

In Asymmetric Multiprocessing, the processors are not equals.

- **Boss Processor (Master):** One processor is designated as the "boss." It controls the entire system.
- **Worker Processors (Slaves):** The other processors are "workers." They look to the boss for instructions or have predefined, specific tasks.

Operational Structure:

- **Centralized Decision Making:** The boss processor handles all critical system decisions, such as **scheduling processes** and **allocating resources**.
- **Task Execution:** The boss assigns work to the worker processors. The workers simply execute the code given to them without worrying about system management.



Pros & Cons:

- **Pro (Simplicity):** It is easier to design the operating system because only one processor accesses the system data structures. This eliminates the need for complex locking mechanisms to prevent processors from fighting over data.
- **Con (The Bottleneck Problem):** The boss processor can become a bottleneck. If the boss is busy handling system interrupts or scheduling, the worker processors may sit **idle** waiting for work. This leads to lower overall system efficiency compared to Symmetric Multiprocessing (SMP).



The “Peer” Model:

- **Definition:** SMP is the most common multiprocessor system. All processors are considered peers (equals); there is no boss- worker relationship.
- **Role:** Each processor is capable of performing all tasks, including executing operating system code and user processes.

Resource Architecture:

- **Private Resources:** Each processor has its own set of registers and a private local cache (L1/ L2).
- **Shared Resources:** All processors share physical memory (RAM) and the system bus.

Key Challenge (Synchronization)

Since multiple CPUs share memory and data structures, the OS must effectively control I/O and memory access to ensure data reaches the correct processor.

Cache Coherency: Hardware must ensure that if one CPU updates data in its cache, the other CPUs see the update.

Usage:

Virtually all modern operating systems- including Windows, Linux, and macOS- provide support for SMP.

Symmetric Multiprocessing (SMP)

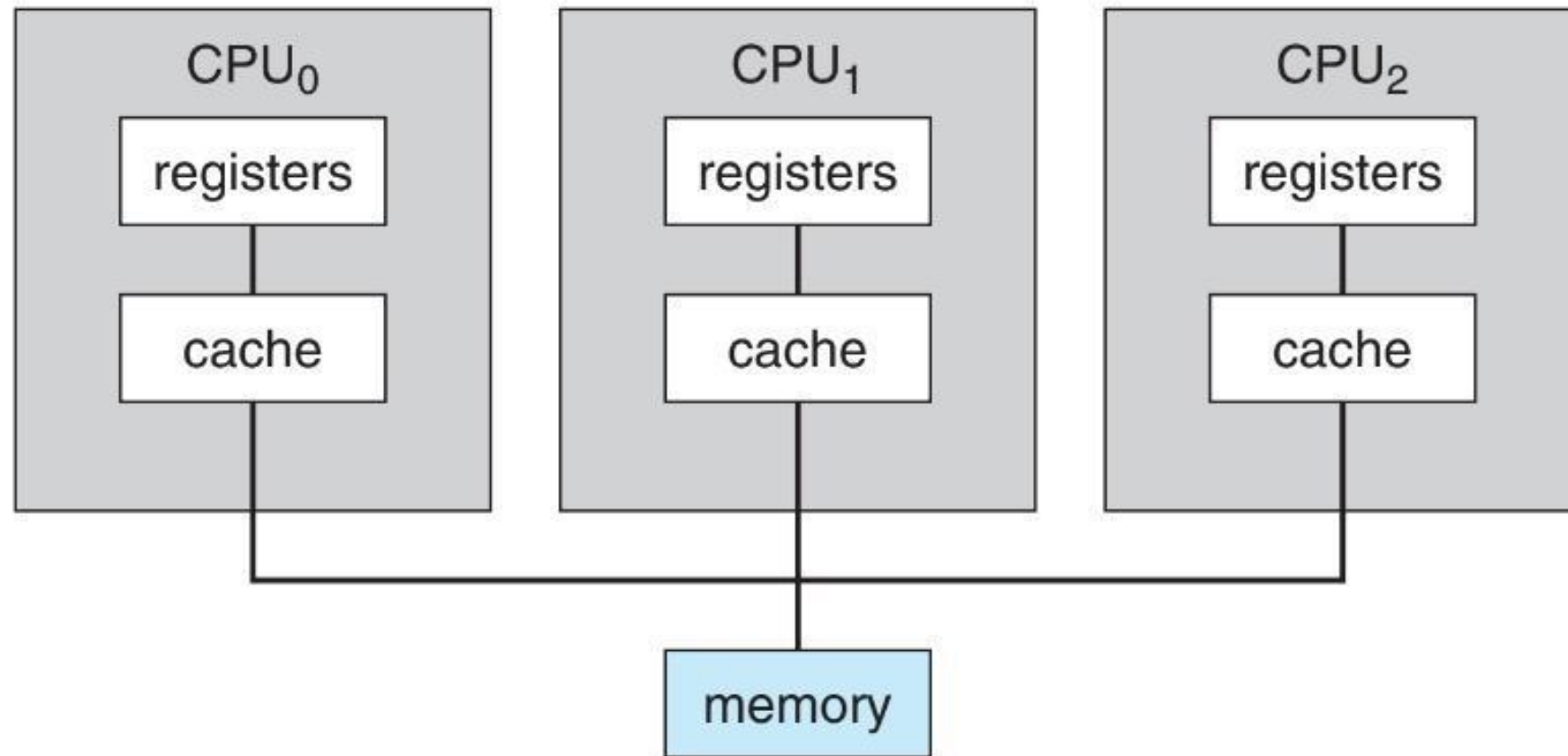


Figure 1.6 Symmetric multiprocessing architecture.



Definition:

A multicore system is a specific type of multiprocessor system where multiple computing cores reside on a single physical chip.

Distinction:

While all multicore systems are multiprocessors, not all multiprocessors are multicore. Older systems used separate physical chips for each CPU, whereas modern systems combine them onto one piece of silicon.



Key Benefits:

- **Efficiency:** On-chip communication is faster than communication between separate chips because signals travel shorter distances.
- **Power Consumption:** A single chip with multiple cores uses significantly less power than multiple separate single-core chips to achieve the same performance.

The OS Perspective: To the Operating System, these cores appear as N standard, separate processors.

Example: A quad-core laptop appears to Windows or Linux as having 4 distinct CPUs available for scheduling.

Implications: This puts pressure on OS designers and application programmers to write multi-threaded code to actually utilize these cores; otherwise, the extra cores sit idle.

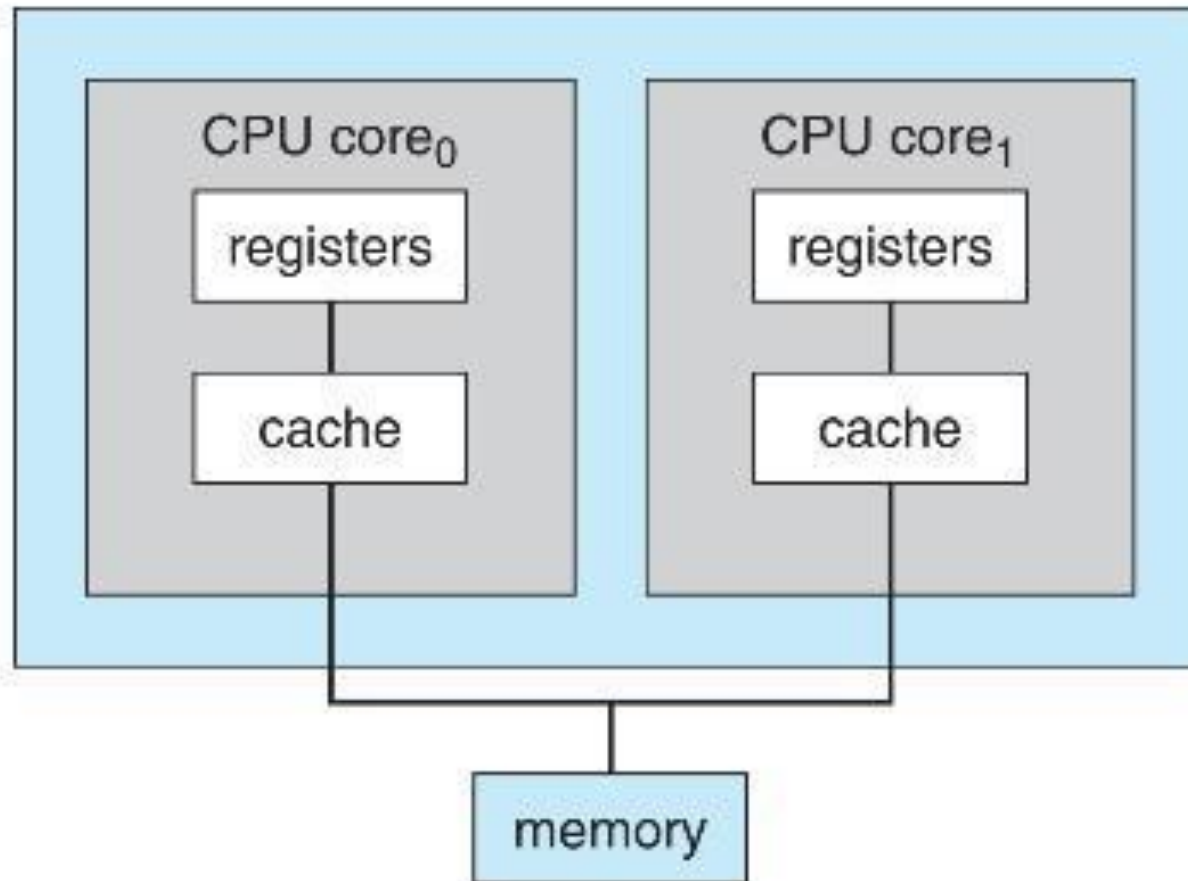


Figure 1.7 A dual-core design with two cores placed on the same chip.



A Clustered System gathers together multiple CPUs.

Difference from Multiprocessors: While multiprocessors are one system with many CPUs, clusters are composed of two or more individual systems (nodes) joined together.

Connection: Nodes are loosely coupled, connected via a network (LAN or fast interconnect like InfiniBand), and typically share storage (via a Storage Area Network or SAN).

Primary Goal:

- High Availability
- Redundancy



Types of Clustering:

- **Asymmetric** (Hot-Standby): One machine runs the application; the other stands by and does nothing but monitor. Simple, but "wastes" the standby hardware.
- **Symmetric**: Multiple hosts run applications and monitor each other. More efficient because it utilizes all available hardware.

High-Performance Computing (HPC):

Clusters can supply greater computational power than a single mainframe.

Parallelization: A technique where a program is divided into separate components that run in parallel on individual computers in the cluster.

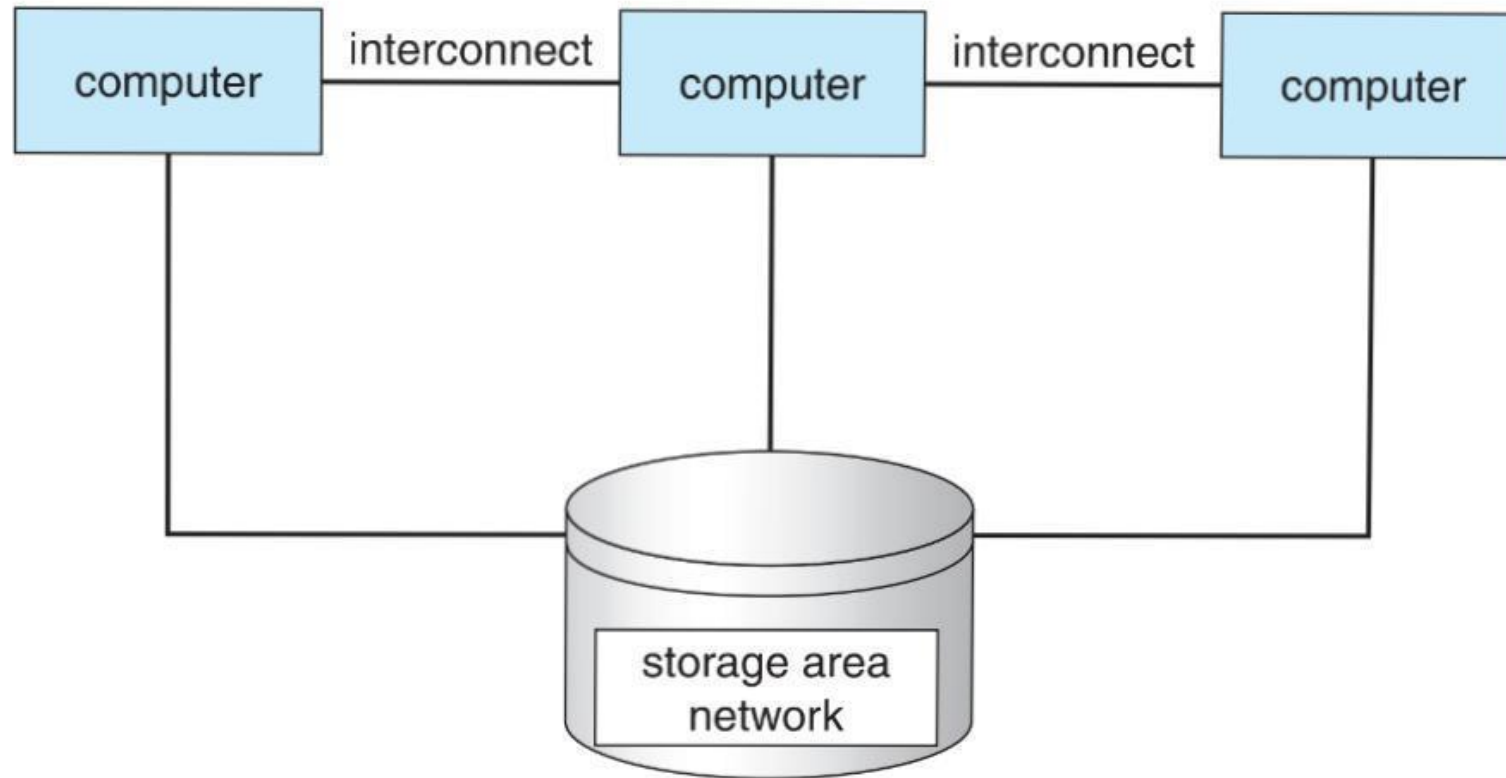


Figure 1.8 General structure of a clustered system.



Operating-System Structure



The Core Challenge: Efficiency

- **The Problem:** A single program cannot keep the CPU and I/O devices busy at all times.
- **The Efficiency Gap:** I/O operations (like reading from a disk or waiting for user input) are significantly slower than CPU processing speeds.
- **The Idle CPU:** In basic systems, if a program waits for user input or disk access, the CPU sits idle.
- **The Goal:** Maximize **CPU Utilization**—ensuring the processor is always doing productive work.



- **Definition:** A technique that organizes "jobs" (code and data) so the CPU always has something to execute.
- The ability of an OS to keep several jobs in memory simultaneously to ensure the CPU always has work.
- **The Job Pool:**
 - Since main memory (RAM) is limited, all waiting processes are stored on the disk in a **Job Pool**.
 - The OS selects a subset of these jobs to reside in memory.



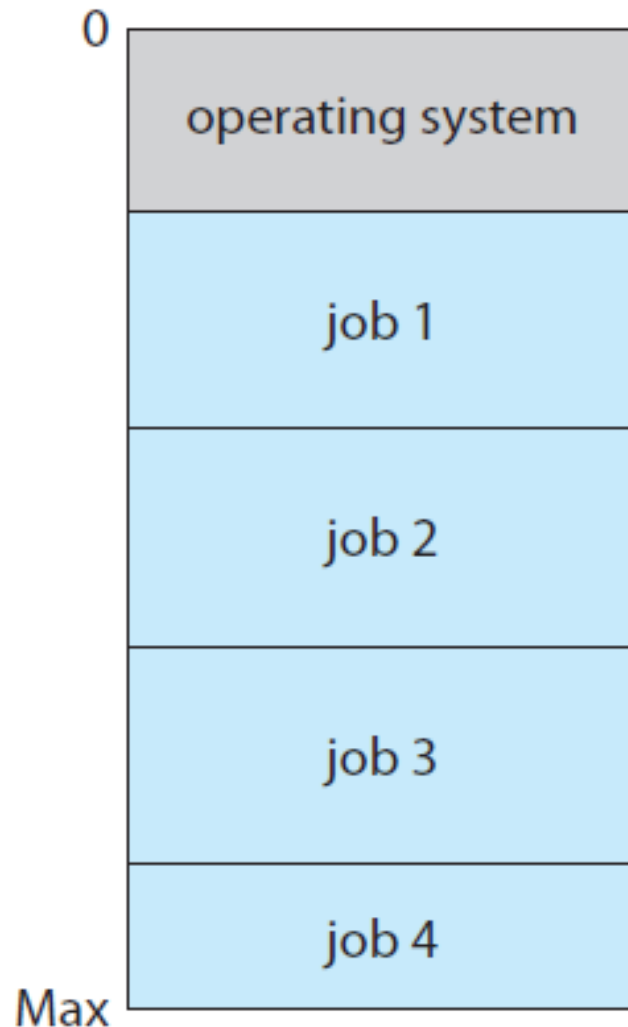
- **Memory Management:** The OS keeps several jobs in memory simultaneously.
- **The Switching Process:**
 - The OS picks one job and starts executing it.
 - The job eventually hits a "wait" state (e.g., waiting for I/O).
 - Instead of waiting, the OS **switches** the CPU to another job in memory.
- **The Cycle:** This continues until the first job is ready again, ensuring the CPU stays busy as long as there is work to do.



The "Lawyer" Analogy

- **Professional Multitasking:** A lawyer doesn't wait for one case to go to trial before starting another.
- **Efficiency in Parallel:** While Case A is waiting for paperwork, the lawyer works on Case B.
 - While Case B is waiting for a court date, the lawyer works on Case C.
- **The Outcome:** As long as there are enough clients (jobs), the lawyer (CPU) is never idle.

Visualizing the Memory Layout



- **The Partitioning of RAM:** To achieve multiprogramming, the Main Memory (RAM) must be divided into distinct sections.
- **The Resident Monitor (OS):** One portion of memory is permanently reserved for the OS itself.
- **User Job Areas:** The remaining memory is split among several active "jobs" (Job 1, Job 2, Job 3, etc.).
- **The Job Pool (Disk):** The "Job Pool" acts as a waiting room for processes awaiting their turn in RAM.
- **Subset Selection:** OS selects a specific subset of jobs from disk and loads them into these memory slots.

Memory layout for a multiprogramming system.



- **The Limitation of Multiprogramming:** While it utilizes resources (CPU, memory, I/O) effectively, it does not prioritize user interaction.
- **Defining Time-Sharing (Multitasking):** A logical extension of multiprogramming where the CPU switches between jobs so frequently that users can interact with each program while it is running.
- **Direct Communication:** Requires an interactive system where users provide instructions via input devices (keyboard, mouse) and receive immediate output.
- **The Goal:** A response time typically less than one second.



The Multi-User Experience

- **Simultaneous Sharing:** Multiple users can share the computer at the same time.
- **High-Speed Switching:** Because user commands are usually short, they require very little CPU time per action.
- **User Perception:** The system switches so rapidly between users that each person feels the entire computer is dedicated exclusively to them.
- **From Program to Process:** A program loaded into memory and executing is formally called a **process**.



Managing the Interactive Flow

- **Handling "People Speeds":** Interactive I/O runs at human speeds (e.g., typing at 7 characters per second), which is incredibly slow for a computer.
 - Instead of sitting idle during this slow input, the OS rapidly switches the CPU to another user's program.
- **Key Management Pillars:**
 1. **Job Scheduling:** Selecting which jobs from the "job pool" enter memory when space is limited.
 2. **CPU Scheduling:** Deciding which job in memory runs first when multiple are ready.
 3. **Memory Management:** Handling several programs residing in memory simultaneously.



- **File System Management:** Necessary for storing user data across collections of disks.
- **Disk Management:** Ensuring the physical storage media is handled efficiently.
- **Resource Protection:** Mechanisms to prevent inappropriate use or unauthorized access to system resources.
- **Deadlock Prevention:** Preventing jobs from getting stuck forever waiting for each other.



Operating-System Operations



The OS is Interrupt Driven:

- The Operating System sits idle (quiet) until something happens.
- If there are no programs to run or I/O to service, the OS does nothing.

Types of Events:

- **Hardware Interrupt:** Sent by hardware devices (e.g., keyboard press, disk finishing a task).
- **Trap (Exception):** A software-generated interrupt.
 - **Caused by Errors:** Division by zero, invalid memory access.
 - **Caused by Requests:** A user program asking for an OS service (System Call).

Handling Events:

- The interrupt-driven nature defines the OS structure.
- For each type of interrupt, specific code segments (Interrupt Service Routines) run to handle that specific action.



Dual-Mode Operation (The Mode Bit)

- To ensure the system is safe, we must distinguish between **Operating-system code** (Trusted) and **User-defined code** (Untrusted/Potentially buggy).
- The computer hardware provides a specific bit, called the **Mode Bit**, to indicate the current mode of execution.

Two Fundamental Modes:

- **Kernel Mode (Bit = 0):** Also called supervisor, system, or privileged mode. This is when the OS is running and has full access to the hardware.
- **User Mode (Bit = 1):** This is when a user application (like a word processor or web browser) is executing.

The Rule:

- When the system is running user code, the system is in **User Mode (1)**.
- When the user needs help (a system call) or an error occurs (trap), the system switches to **Kernel Mode (0)**.



Transitions Between Modes (The Cycle)

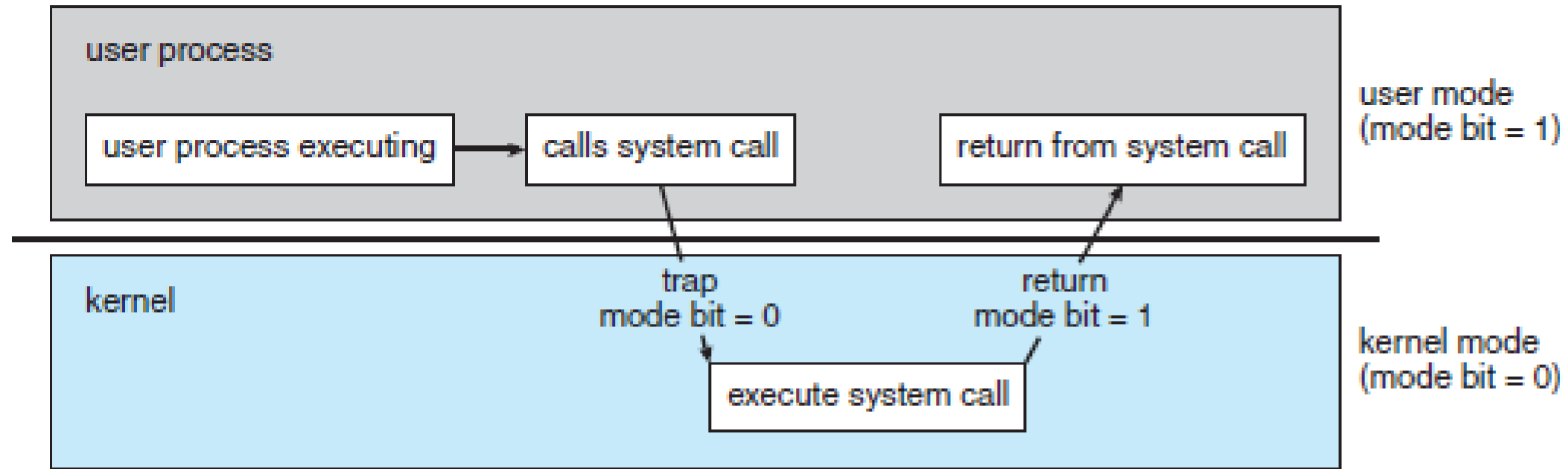


Figure 1.10 Transition from user to kernel mode.



Transitions Between Modes (The Cycle)

- At boot time, the hardware starts in **Kernel Mode (0)**.
- The OS loads, sets up, and then starts user applications in **User Mode (1)**.

The Transition Cycle (See Figure 1.10):

1. **User Mode:** The User process executes.
2. **Event Occurs:** A **System Call** (request for help) or a **Trap** (error/interrupt) occurs.
3. **Switch to Kernel:** The hardware changes the mode bit from **1** $\rightarrow$ **0**.
4. **Execute:** The OS gains control, executes the service routine in Kernel Mode.
5. **Return:** Before returning control to the user program, the OS switches the mode bit back from **0** $\rightarrow$ **1**.

Key Takeaway: Whenever the Operating System gains control of the computer, it is in Kernel Mode.



Privileged Instructions

Definition: Machine instructions that may cause harm if executed incorrectly.

Examples:

- I/O control (accessing disk/network).
- Timer management.
- Interrupt management (turning signals on/off).

The Hardware Rule: Privileged instructions can be executed **only** in Kernel Mode (Mode Bit = 0).

Violation Mechanism: If a user program (in Mode 1) attempts to execute a privileged instruction:

- It treats the instruction as illegal.
- It issues a **Trap** to the Operating System, which typically terminates the program.



Beyond Dual-Mode: Modern systems often need more than just "User" and "Kernel" modes.

Virtual Machine Manager (VMM):

- Software that manages Virtual Machines (VMs).
- Needs more privilege than a standard user program (to manage VMs) but fewer privileges than the full hardware kernel (to ensure safety).

Hardware Support:

- CPUs may support **multiple privilege levels** (e.g., Intel 64 supports **4 distinct rings/levels**).
- This allows the VMM to run in a protected "middle" mode, keeping the system secure while managing guest operating systems.



Why Dual-Mode is Critical:

- **The Risk:** Without hardware protection (no Mode Bit), a user program can wipe out the OS or write to devices directly.
- **Historical Example (MS-DOS):** Written for the Intel 8088, which had no mode bit. A single crashing program could freeze the entire system or corrupt the disk.
- **Modern Systems:** Windows, Linux, and macOS use the Mode Bit to ensure a user bug *only* crashes the application, not the computer.

Handling Violations:

- **Detection:** If a program tries to run a privileged instruction or access forbidden memory, the hardware detects the error.
- **The Trap:** The hardware immediately sends a **Trap** to the Operating System.

OS Action:

1. Terminates the program abnormally.
2. Generates an error message.
3. Creates a **Memory Dump** (a file containing the program's memory state) for the programmer to debug later.



The Timer (Ensuring OS Control)

The Problem:

- How does the OS prevent a user program from getting stuck in an **infinite loop** or hogging the CPU forever?
- If the OS passes control to the user, how does it get it back?

The Solution: A Timer: A hardware device set to interrupt the CPU after a specific period (e.g., 1 millisecond).

- **Mechanism:**

1. **Clock:** Ticks at a fixed rate.
2. **Counter:** The OS sets a value. Every tick, the counter decrements.
3. **Interrupt:** When the counter reaches 0, the hardware sends an interrupt, and control returns to the OS.

Privileged Operation: The instruction to *load* the timer is **Privileged**. A user program cannot turn off the timer to extend its own life.

Usage:

- **Time Sharing:** Ensures no user takes more than their fair share of time.
- **Time Limits:** If a program runs too long (e.g., 7 minutes), the OS terminates it.